

Digital Fashion Passport

Project plan and technical overview

ETHGlobal hackathon · Solidity + ERC-721 · deployed to the Sepolia testnet · Amelie Gonzalez Capdepon

Live at digital-fashion-passport.vercel.app · contract `0x67ad5c6b250f08b1cfd749cc371cdd140d81a61f`

The idea

I love fashion and I love blockchain. I wanted to know how the two could actually intertwine instead of just sitting next to each other.

So that is where this started. A system where a physical piece of clothing carries a record that belongs to whoever owns it, and that looks good enough that someone would want to use it. Most blockchain products manage neither of those. I set out a plan, did the research, and built the smallest version that works end to end.

The project gives physical fashion items a digital identity that follows them for their whole lifetime. You register a piece with its brand, its model, and a link to whatever documents back it up. That record is minted as an NFT held by your wallet. When the piece is resold, the passport goes with it, so the ownership history stays verifiable.

The point is a record that connects a physical garment to on-chain ownership, so anyone can check authenticity without having to trust a company to keep a database honest.

Who it is for

Upper and middle class buyers of high-value fashion.

- Nobody is putting a Primark t-shirt on a blockchain. The value is too low and the gas is not worth it.
- A Gucci bag or a high-end jacket is different. These are pieces people save up for, insure, resell, and care about proving.
- The burglary case is the clearest one. If a stolen piece gets taken to a pawn shop to be resold, the shop can check the chain and see who actually holds the passport.
- The resale case is the everyday one. A buyer on a secondhand marketplace can verify a piece is what the seller says it is before paying for it.

What the passport represents

In the first brainstorm I was going to store the owner's name next to their wallet address and the product details. I dropped the name.

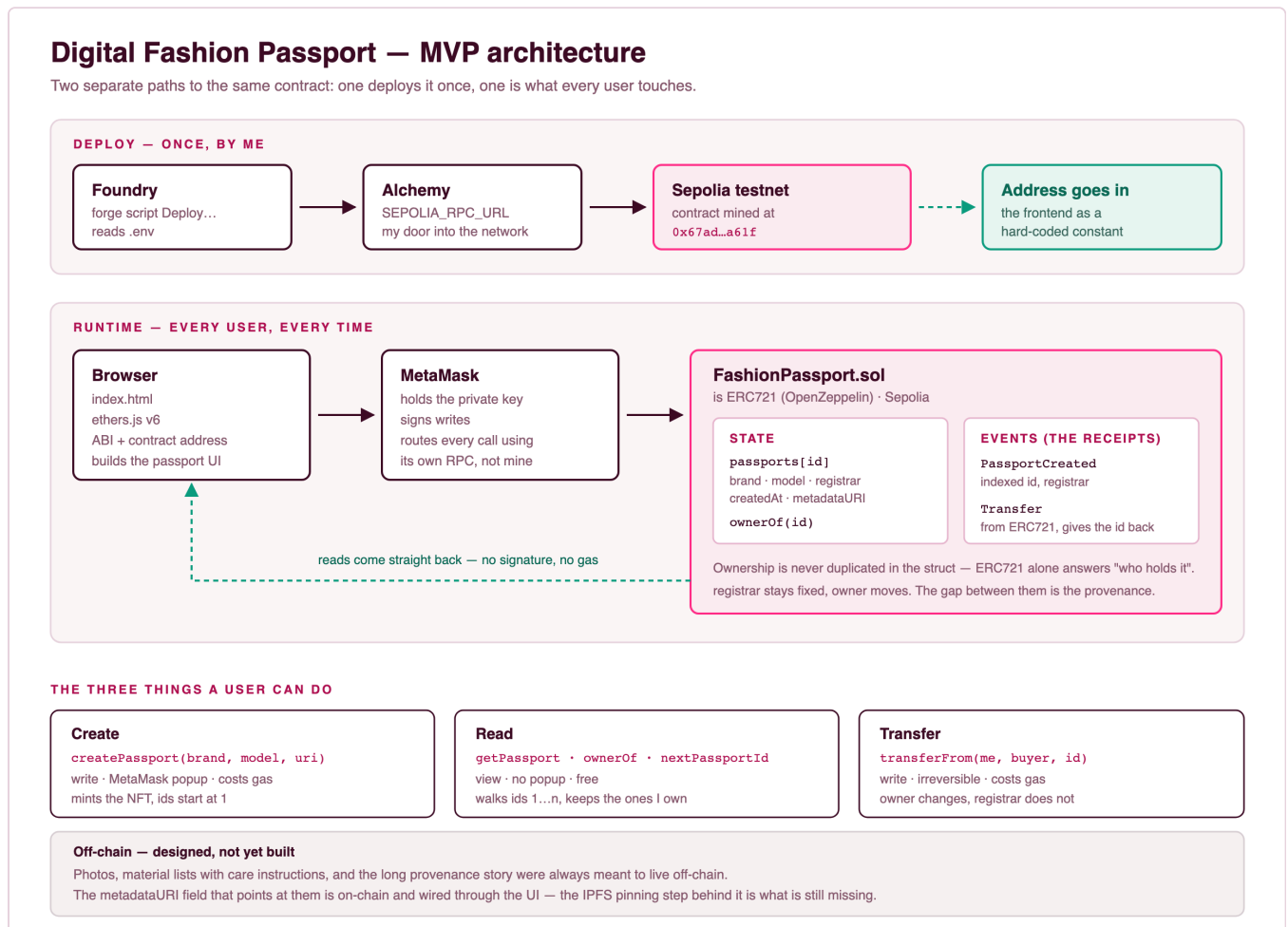
The wallet address does the job on its own. It proves ownership has stayed continuous without putting a legal identity onto a public record, so you keep your anonymity and can still prove the piece is yours. You

have to show the wallet to claim a passport, and that is the security part. You cannot claim something you cannot sign for.

This is why the contract has no name field, and why there is no login page. The wallet is the account.

Architecture

The diagram below splits the two paths that people usually mix up. One is how the contract got deployed, which happened once, from my laptop. The other is what a user actually touches.



Deploy path and runtime path, and the three actions a user can take.

Alchemy is not part of the live app. It gave me the RPC endpoint that Foundry used to push the deployment transaction to Sepolia. Once that was mined the contract had an address, and the address became a constant in the frontend. At runtime the browser talks to MetaMask, and MetaMask reaches the network through its own provider. If Alchemy went down tomorrow the site would carry on working.

How it works

The contract

FashionPassport.sol inherits ERC-721 from OpenZeppelin, so a passport is an NFT. I picked that over writing ownership by hand. ERC-721 already handles transfers, approvals and ownership lookups, it has been audited far more than anything I could write in a weekend, and it means the passports show up in MetaMask without any extra work.

Each passport is a struct in a mapping, keyed by an id that starts at 1.

Field	Type	What it holds
brand	string	The maison. Gucci, sacai, Cartier
model	string	The specific piece, like Horsebit Loafers
registrar	address	Who first registered it. Never changes.
createdAt	uint256	Block timestamp of registration
metadataURI	string	Link to everything stored off-chain

There is no `owner` field in that struct, and that is on purpose. ERC-721 already tracks ownership in its own table. Storing it twice would give me two answers that could disagree with each other. The current owner comes from `ownerOf(id)`. The registrar comes from the struct. After a resale those two stop matching, and that mismatch is the provenance.

Events as receipts

The contract fires an event every time a passport is created. ERC-721 fires its own `Transfer` event on every mint and every resale.

Events are how the frontend finds out the id of a passport it has just made. A transaction cannot hand a value back to the browser, because at the moment the call returns it has only been broadcast. It has not run yet.

```
event PassportCreated(uint256 indexed id, address indexed registrar, string brand, string model);
```

`indexed` on `id` and `registrar` makes those fields searchable, so the ownership history can be rebuilt from the logs. Storing an array on-chain instead would cost gas on every single write.

Testing

I wrote the tests in Foundry before deploying anything, and ran them on Anvil, which is a local chain on my own machine with fake money. There are five. They check that a passport stores what it was given, that owner and registrar get set correctly, that ids increment from 1, that a transfer moves the owner but leaves the registrar alone, and that someone who does not own a passport cannot transfer it.

That last one is the security test. It asserts the transaction reverts, so it proves the contract refuses rather than proving it succeeds.

The frontend

One HTML page using ethers.js v6 from a CDN. It holds the contract address and the ABI, which is the translation table between readable function calls and the bytes the chain wants. MetaMask injects itself into the page, ethers wraps it as a provider, and asking that provider for a signer is what makes the popup appear.

Writes cost gas and open MetaMask. Reads are free and open nothing at all. Keeping that difference visible in the interface, so that a transfer asks you to confirm and a search does not, is most of what makes the thing feel trustworthy.

On-chain and off-chain

The split follows the same logic the EU uses for its own product passports. Keep the minimum on the expensive permanent layer, and put the bulky material somewhere cheap that the permanent layer points at.

On-chain (built)	Off-chain (designed, not yet built)
brand, model, registrar, createdAt, metadataURI, plus ERC-721 ownership and the event log	Photos of the piece, material lists with care instructions, the long provenance story, and anything else that is long or likely to change

Where this actually stands: `metadataURI` is on-chain and wired through the interface, so the structure is there. The pinning step is not. Right now an attached photo gets read into the browser and displayed, and it disappears on refresh. Materials, care instructions and the provenance story were always part of the plan, and they are the first thing I would build with more time.

Research

I checked whether this already existed before building it. It turns out the European Union has been working on the same problem for physical products.

Their Digital Product Passport is not one central database. The data lives across thousands of independent locations, run by the businesses themselves. The EU Central Registry behaves more like a card catalogue than a warehouse. It holds high-level metadata and unique reference keys, so it knows which products exist and who manufactured them, which is enough for customs and market inspectors to confirm a product has a valid passport. The detailed lifecycle data stays with the manufacturers: exact material weights, carbon footprints, supply chain origins, repair manuals.

Which is the same shape as this project. A small authoritative record pointing at richer data held somewhere else. I found it after I had already made the same decision, which was reassuring. The registry currently covers batteries, steel, textiles and electronics, and textiles is where this would sit.

Sources

- [CIRPASS, DPP System Architecture \(D3.2\)](#)
- [European Commission, Digital Product Passport Registry](#)
- [European Commission, Digital Product Passport overview](#)
- [CIRPASS, DPP System Roadmap \(D3.4\)](#)
- [One Click LCA, What is a Digital Product Passport](#)

What the MVP does

Four features, all working against the deployed contract on Sepolia.

Feature	Contract call	Notes
Connect wallet	<code>none</code>	MetaMask only, and it checks the network is Sepolia
Register an item	<code>createPassport</code>	Write. Mints the NFT and returns a new id.
View passport list	<code>getPassport, ownerOf</code>	Read. Walks ids 1 to n and keeps the ones this wallet owns.
Transfer	<code>transferFrom</code>	Write. Irreversible, so it asks you to confirm.

Price history with Chainlink PLANNED

The passport records who owned a piece and when. It does not record what it was worth. That is the next gap I want to close, and I want to be clear about why it is not in this version.

The idea

When you register a piece you would declare what you paid for it. When you transfer it you would declare the sale price. The contract stores both. A Chainlink price feed supplies the ETH/USD rate, so the interface can show what a piece was worth at the moment it changed hands, next to what that is worth today.

The example that made me want it: you buy a bag for 1,000 USDC. Time passes, the market moves. Because the value is tied to a live feed instead of a number someone typed into a database, the passport can show that the same holding is now worth somewhere around 1,500. And it can show that to a buyer who has no reason to take the seller's word for anything.

That turns the passport from a record of custody into a record of value over time, which is what makes it interesting to the person deciding what to pay. I did a Chainlink bootcamp recently and I would like to apply it here properly.

How it would work

Sepolia has an ETH/USD aggregator. The contract calls `latestRoundData()` on it and divides the answer by $1e8$ to get a usable figure. Reading the feed is the straightforward half.

Why it is not in this version

The hard half is that my contract stores no value at all. No purchase price, no declared worth, so there is nothing for a price feed to convert. Adding one means changing the struct. Changing the struct means deploying a new contract at a new address.

Option	Verdict
Show a live ETH/USD figure somewhere on the page	Bolted on. It is a price ticker, not an integration, and it reads as one.
Add a price field to the contract and redeploy	Meaningful, but a new contract address strands every passport already minted, and means re-minting and re-testing everything.
Push the price into <code>metadataURI</code>	Avoids the redeploy, but puts a number that should be verifiable into an unverifiable string.

Doing it properly means a redeploy, and a redeploy throws away the deployed address, the transaction history already sitting on Etherscan, and the passports currently in my wallet. I would rather submit something that works end to end than redeploy the day before the deadline for a field I cannot demonstrate carefully.

Ownership history is on-chain, but value is not. The next version records a declared sale price on transfer and uses Chainlink's ETH/USD feed to show what a piece was worth when it changed hands versus today. I left it

out because it needs a contract change, and shipping something that works mattered more than adding a field I could not test properly.

Known limitations

- The off-chain layer is not built. Photos, materials and provenance text have a field pointing at them and nothing behind it yet.
- No value is recorded, for the reasons above.
- MetaMask only. The app reads `window.ethereum` directly instead of using a wallet connection library, so other wallets cannot connect.
- Reading the list walks every id from 1 upwards. Fine for a demo, and it gets slower as the collection grows. The proper fix is to read the event log instead, which is what the indexed fields were added for.
- MetaMask shows a security warning the first time someone connects to the deployed site at digital-fashion-passport.vercel.app. That is normal behaviour for a domain it has not seen before, and it is doing exactly what it should. It is still the first thing a new visitor sees, which does not help a project whose whole subject is trust. A custom domain with some history behind it would reduce it.
- Anyone can register any brand. The contract does not check that whoever registers a Gucci bag has anything to do with Gucci. That is why `registrar` is stored. It records who made the claim without endorsing it.

Next up

I am carrying on with this past the hackathon. These four are what I am building next.

- **Share-a-passport links**, so a buyer can verify a piece before they pay. At the moment you have to connect a wallet to see anything, which is backwards for the person who most needs convincing. A public read-only link for one passport turns this from a private collection into something you could actually use in a resale conversation.
- **ENS names instead of raw addresses**, shown in the passport and accepted in the transfer box. A name is much easier to check against what a seller told you than forty hex characters. Frontend only, no contract change.
- **Chainlink ETH/USD**, so you can see what a piece was worth when it changed hands, as set out above. This one needs the contract change and the redeploy, so it leads a version that is allowed to break the current address.
- **Category on-chain**, moving it out of the interface and into the struct, so pieces can be filtered by type and the data means the same thing to anyone reading the contract rather than just to my frontend.

Further out: building the off-chain layer properly with IPFS pinning, letting brands verify themselves so a passport registered by Gucci reads differently from one registered by a member of the public, supporting wallets other than MetaMask, a landing page and an about page so a first-time visitor understands the project before being asked to connect anything, and a cap on how many passports one address can register as a basic defence against spam.

Contract deployed at 0x67ad5c6b250f08b1cfd749cc371cdd140d81a61f on the Sepolia testnet.